



UNIVERSITI MALAYA

**WQD7008: PARALLEL AND DISTRIBUTED COMPUTING
ALTERNATIVE ASSESSMENT - II
DR. BRYAN RAJ A/L PETER JABARAJ**

Name	Matric Number
WAN NURUL ADIBAH BINTI WAN TARMIZI	24062857
YUHAN SILVIAN	24077279
BAIZID YALDRAM	23117259
NUR IZZATI BINTI JUHARI	24075056
MADINA SURAYA BINTI ZHARIN	24071524

TABLE OF CONTENT

Abstract	4
1.0 Introduction	4
1.1 Background Study	4
1.2 Problem Statement	4
1.3 Objectives	5
2.0 Distributed and Parallel Computing Justification	5
3.0 Proposed Architecture Design	6
3.1 AWS S3 Data Lake - Raw Data Storage Layer	7
3.2 AWS EMR - Distributed Processing Layer	7
3.3 AWS S3 - Processed Data Storage Layer	7
3.4 AWS Redshift & QuickSight - Analytics & Visualization Layer	7
4.0 Architecture Design Implementation	8
4.1 Data Ingestion and Storage	8
4.2 Data Processing Layer	8
4.3 Data Analytics and Visualisation	20
5.0 Results & Observations	21
6.0 Discussion	21
7.0 Conclusion	21
References	22

Abstract

The New York City Metropolitan Transportation Authority (MTA) introduced a fare-free bus pilot in 2023 to improve accessibility across five selected routes. The pilot generated large-scale, continuous real-time data from GPS trackers, automated passenger counters, and traffic sensors across thousands of buses. Traditional single-system processing methods are insufficient for analysing such high-velocity and geographically distributed datasets, which are essential for service evaluation and operational optimization. This study explores the use of distributed and parallel computing frameworks to manage and analyse real-time transit data efficiently. A scalable architecture incorporating technologies such as Apache Spark, Hadoop, Kafka, and edge computing is proposed to support real-time decision-making, route optimization, congestion detection, and performance monitoring. The findings aim to demonstrate how distributed computing can enhance processing speed, scalability, and reliability while supporting future expansion, including potential implementation of citywide fare-free public transit.

1.0 Introduction

1.1 Background Study

In September 2023, the MTA launched a pilot program offering free bus rides on five routes, one in each borough of New York City. This initiative aimed to reduce economic barriers to transportation and improve mobility for underserved populations. The program concluded on August 31, 2024, due to budget constraints and concerns regarding long-term revenue loss and fare evasion. Although the pilot ended, discussions continue regarding the feasibility of expanding fare-free transit citywide, with political support from leaders such as Mayor-elect Zohran Mamdani.

The pilot generated substantial real-time operational data from diverse and distributed sources, including bus movement, passenger demand patterns, congestion hotspots, and service delays. Modern public transportation systems rely heavily on advanced computing to analyse such data, improve operational efficiency, and support predictive service management. Distributed and parallel computing technologies are increasingly relevant as they enable scalable, low-latency processing of high-volume streaming data, the capabilities that traditional centralized analytical systems lack.

1.2 Problem Statement

The fare-free bus pilot in New York City resulted in massive and continuous data generation from thousands of buses equipped with sensors and communication systems. Efficiently analysing these heterogeneous, high-velocity data streams is essential for assessing service accessibility, operational performance, congestion forecasting, and passenger experience. However, conventional single-node computing infrastructures are inadequate for processing such large-scale, geographically distributed real-time datasets. Therefore, an optimized distributed and parallel computing framework is required to support timely data processing, system monitoring, real-time routing decisions, and scalability should the fare-free model expand citywide.(Hassan et al., 2025)

1.3 Objectives

This research aims to fulfill the following objectives:

1. To analyse high-frequency real-time bus operational data, including GPS coordinates, passenger load, delays, and traffic conditions, performance patterns.
2. To develop a scalable distributed computing framework capable of processing continuous and geographically dispersed data streams generated by the transit system.
3. To evaluate the performance, scalability, and responsiveness of the proposed distributed system in supporting real-time analytics, route optimization, and operational decision making.

2.0 Distributed and Parallel Computing Justification

Parallel computing is a computing standard in which problems will be divided into smaller tasks that will be executed simultaneously while distributed computing involves multiple interconnected nodes that collaborate over a network to share resources and process workloads beyond the capacity of a single machine (Czarnul et al., 2025).

For the NYC Bus System Optimization problem, parallel and distributed computing enable scalability through load balancing, where workloads are distributed across multiple nodes to prevent system overload. As data volume increases, horizontal scaling allows additional nodes to be added, reducing response time and mitigating processing bottlenecks (Hoque et al., 2024).

The bus system generates high-volume, time-sensitive data streams that require near real-time ingestion and analysis. Single-node systems are inadequate for such workloads, particularly under failure conditions. Distributed architectures improve fault tolerance by allowing processing to continue even if individual nodes fail, ensuring uninterrupted data flow and analysis (Hoque et al., 2024). In the case of the NYC Bus System, if thousands of buses send data continuously, other servers should automatically take over the task so that the data stream does not stop. Additionally, cloud storage services such as S3 provide high fault tolerance, allowing failed tasks to be re-executed so that route performance and waiting time analyses remain accurate.

Additionally, the geographically distributed nature of data across multiple boroughs benefits from distributed processing, which reduces latency by enabling concurrent execution across nodes. Cloud-based deployment further enhances elasticity by dynamically allocating resources during peak travel periods and releasing them during off-peak periods, improving overall cost efficiency (Hoque et al., 2024).

3.0 Proposed Architecture Design

The production architecture is designed to handle continuous, high-volume data streams from city-wide bus operations. It uses managed AWS services that support large-scale distributed processing, real-time ingestion, and analytical workloads. The architecture consists of five major layers: ingestion, raw storage, distributed processing, processed storage, and analytics/visualization.

3.1 AWS Kinesis - Data Ingestion Layer

The data ingestion layer utilizes Amazon Kinesis to capture and stream real-time operational data generated by NYC buses, including GPS locations, passenger counts, and traffic conditions. Kinesis enables scalable and low-latency data ingestion from distributed sources and continuously delivers incoming data streams to the Amazon S3 data lake for downstream distributed processing.

3.2 AWS S3 Data Lake - Raw Data Storage Layer

Raw datasets ingested from Kinesis are persisted in Amazon S3, which serves as the central data lake for the architecture. S3 provides highly durable, cost-efficient, and infinitely scalable object storage, making it suitable for retaining high-volume transit telemetry, historical wait-time logs, and other operational datasets. By decoupling storage from compute, S3 enables flexible access patterns for multiple downstream consumers, including Spark, Redshift, and future analytical workloads.

3.3 AWS EMR - Distributed Processing Layer

The core computation layer is powered by Apache Spark running on Amazon EMR. EMR provides a distributed cluster consisting of a master node and multiple worker nodes, enabling horizontal scaling for large-scale data processing tasks. Spark's in-memory execution model provides high performance for iterative analytics, including:

- a. borough-level wait-time aggregation
- b. route efficiency scoring
- c. congestion and delay pattern detection
- d. temporal trend analysis across multiple time windows

EMR automatically manages cluster provisioning, node failures, resource allocation, and parallel job execution, allowing Spark workloads to run efficiently without manual cluster administration.

3.4 AWS S3 - Processed Data Storage Layer

Outputs generated from Spark jobs such as aggregated statistics, route performance indicators, and cleaned feature datasets are written back to Amazon S3 for persistent storage. This processed data layer forms the foundation for downstream analytical queries and dashboard visualizations. Persisting processed outputs in S3 also enables auditability, reusability, and cost-effective long-term retention.

3.5 AWS Redshift & QuickSight - Analytics & Visualization Layer

Processed datasets stored in S3 are efficiently ingested into Amazon Redshift using the optimized 'copy' command. Redshift is a fully managed cloud data warehouse utilizing columnar storage and a massively parallel processing (MPP) engine, allowing transport planners to perform multi-dimensional analysis on historical and near-real-time performance metrics at scale.

For visualization, Amazon QuickSight connects directly to Redshift to generate interactive dashboards and operational insights. This enables stakeholders to monitor route-level performance, peak-hour delays, borough comparisons, and overall service reliability through accessible, real-time visual reporting.

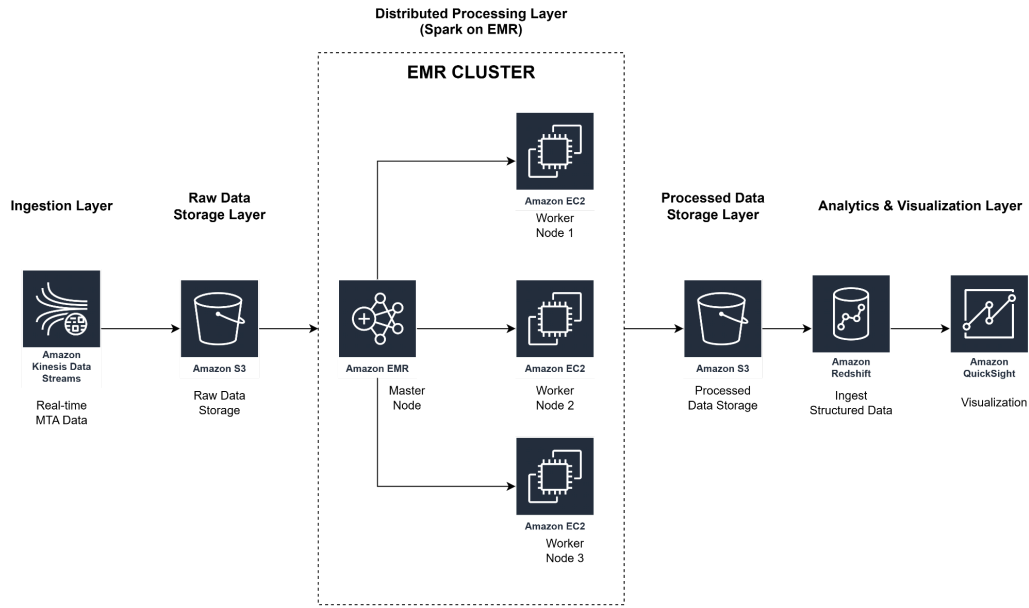


Figure 1. Production Distributed Architecture using Kinesis, S3, EMR, Redshift, and QuickSight.

4.0 Architecture Design Implementation

This section describes the implementation of the proposed distributed and parallel computing architecture for analysing New York City MTA bus operational data. The system was implemented using an AWS Learner Lab environment, which provides limited cloud resources for academic purposes.

Due to resource and service restrictions in the Learner Lab environment, not all components of the proposed production architecture could be implemented exactly as designed. Certain services and configurations were adapted while preserving the core distributed computing concepts, including parallel data processing, horizontal scalability, and fault tolerance.

Despite these constraints, the implementation remains aligned with the logical architecture presented in Section 3 and effectively demonstrates how distributed computing can be applied to large-scale transportation data analytics. The following subsections describe each implemented layer in detail, beginning with data ingestion and storage.

4.1 Data Ingestion and Storage

In this stage, an Amazon S3 bucket, “pdcbucket” was provisioned within the AWS Learner Lab environment to function as the primary Raw Data Storage layer. To ensure logical separation between input and output streams, a directory hierarchy comprising the below folders were created as shown in Table 1. The dataset was subsequently uploaded to the raw/ directory, emulating the initial data ingestion phase of the proposed system architecture.

Table 1: Directory structure

Directory Path	Functional Role	Purpose
raw/	Data Ingestion Layer	The directory acts as the landing zone for unprocessed source data. The directory simulates the entry point for the pipeline where the original dataset is stored.

scripts/	Analytics Logic Layer	The directory houses the PySpark scripts used to perform distributed analytics on the data. Centralizing the scripts here allows the processing clusters to dynamically retrieve and execute the required logic for data transformation.
processed/	Transformation Output Layer	The directory serves as the repository for processed data that has undergone the required transformation. It also acts as the source for downstream analytics and reporting.

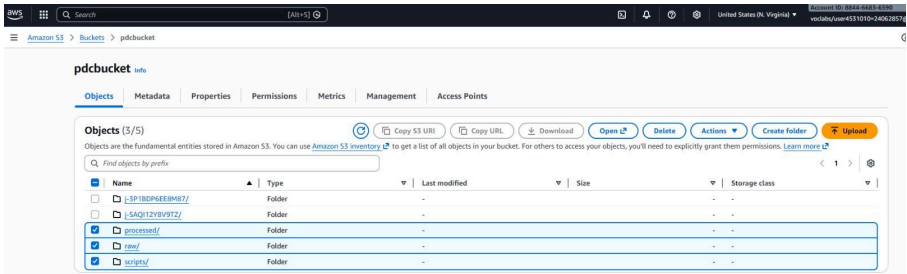


Figure 2. Creation of “raw/”, “processed/ “ and “scripts/” directories within the Amazon S3 bucket.

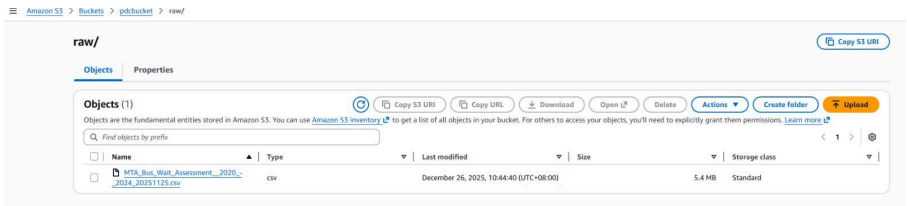


Figure 3. Upload of the source dataset into the raw/ directory of the Amazon S3 bucket.

4.2 Data Processing Layer

I. AWS EMR Cluster Configuration and Setup

This stage represents the implementation of the distributed processing layer using Amazon Elastic MapReduce (EMR) with Apache Spark. This stage represents the core computational engine of MTA data analytics pipeline, designed to process large-scale transportation data through parallel distributed computing. The implementation adheres to the architectural specifications outlined in our system design, featuring a one master node, two worker node configuration for optimal resource utilization and processing efficiency.

The EMR cluster was provisioned using the AWS Management Console with the identifier mta-emr-cluster. Technical specifications included Amazon EMR release 6.15.0 with Apache Spark 3.5.1 and Hadoop 3.3.6 as the foundational frameworks. The hardware configuration precisely matched our architectural requirements: one master node and two core worker nodes, each utilizing m5.xlarge instances (4 vCPUs, 16 GiB memory). Network configuration employed VPC with subnet in the us-east-1a availability zone, while security was managed through IAM roles EMR_DefaultRole for service operations and EMR_EC2_DefaultRole for EC2 instance permissions, with additional security through designated security groups.

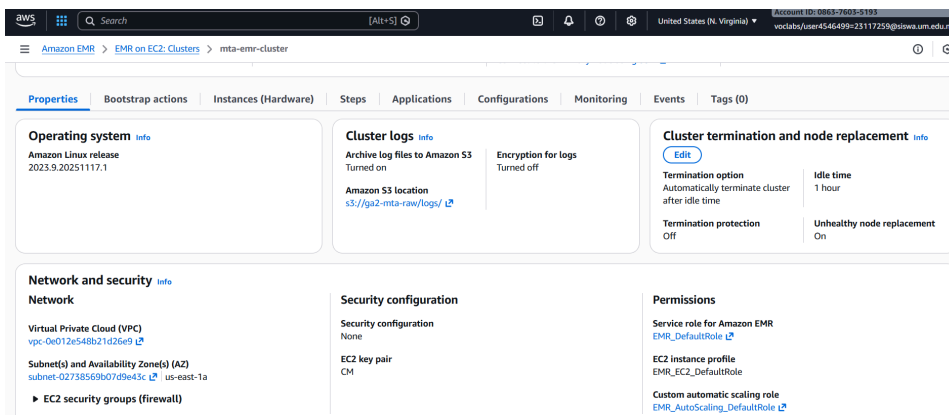


Figure 4. EMR cluster main page

The cluster initialization followed a structured three-phase process over approximately 15 minutes. During the provisioning phase, AWS allocated the EC2 instances and configured networking components, visible in the console as instances transitioning from "requested" to "allocated" status. The bootstrapping phase involved automated installation of Spark, Hadoop, Hive, and necessary dependencies, with the master node typically completing this process before worker nodes. Finally, the service initialization phase launched distributed system components including HDFS NameNode, YARN ResourceManager, and Spark services, in the cluster achieving "WAITING" status indicating full operational readiness for job execution.

Instance groups (2) Info							
With the instance groups configuration, each node type consists of the same instance type and the same purchasing option for instances: On-Demand or Spot.							
Find instances by status ▼ Find resources by ID or type; or search for text within loaded results							
Type and name	ID	Status Last state change reason	Instances	Purchasing option and p...	EBS size (GiB)	EC2 Instance ID i	
☐ Primary	ig-24QT31804J1FH	Running	1	On-Demand	-	-	
☐ Core	ig-3PWY5F5BNA5B0	Running	2	On-Demand	-	-	

Figure 5. EMR and EC2 clusters

Post deployment verification confirmed the exact architectural specification: one primary (master) node and two core (worker) nodes, all utilizing m5.xlarge instance types. This configuration provides a balanced compute environment with 12 total vCPUs and 48 GiB of aggregate memory available for distributed processing tasks. The master node serves as the cluster coordinator, managing resource allocation through YARN and maintaining cluster metadata, while the two worker nodes execute parallel data processing tasks as Spark executors, enabling horizontal scalability for large-scale MTA data processing.

Uniform instance groups

Primary

Choose EC2 instance type

m5.xlarge
4 vCore 16 GiB memory
EBS only storage On-Demand price: -
Lowest Spot price: -

Actions ▼

☐ Use high availability
Launch highly available, more resilient cluster with three primary nodes on On-Demand Instances. This configuration applies for the lifetime of your cluster. [Learn more](#)

► Node configuration - optional

Core

Choose EC2 instance type

m5.xlarge
4 vCore 16 GiB memory
EBS only storage On-Demand price: -
Lowest Spot price: -

Actions ▼

► Node configuration - optional

Figure 6. EMR clusters hardware

II. S3 Storage Integration and Data Flow Architecture

Integration with the Stage 1 S3 storage layer was validated through multiple verification steps. The cluster's logging configuration was set to archive logs to `s3://ga2-mta-raw/logs/`, confirming write permissions and connectivity. The S3 bucket `ga2-mta-processed`, previously configured with `raw/`, `processed/`, and `scripts/` directories was verified as accessible from the EMR cluster, establishing the complete data flow architecture: raw MTA data ingestion to S3, distributed processing through EMR, and processed output storage back to S3. This integration completes the data pipeline's foundation, enabling subsequent stages (Redshift and QuickSight) to access processed data for analytics and visualization.

III. AWS EMR Spark Job Development and Execution

A PySpark application was executed on Amazon EMR to perform distributed batch processing on the MTA Bus Wait Assessment dataset. The job reads raw CSV data from Amazon S3, groups records by borough, and computes average values for key operational metrics. The resulting aggregated dataset is written back to Amazon S3 under the processed data zone (`processed/`) in CSV format with headers, enabling downstream ingestion into Amazon Redshift for analytical querying.

The following PySpark applications were uploaded to Amazon S3 and executed on Amazon EMR to perform distributed analytics on the MTA Bus Wait Assessment dataset, covering borough-level aggregation, route-level efficiency scoring, congestion and delay hotspot identification, and temporal trend analysis.

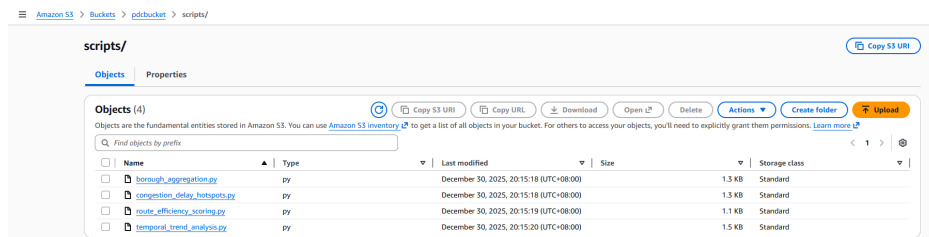


Figure 7. PySpark application scripts

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, avg, sum, when
3
4 spark = SparkSession.builder.appName("borough_aggregation").getOrCreate()
5
6 INPUT = "s3://pdcbucket/raw/"
7 OUTPUT = "s3://pdcbucket/processed/borough_aggregation/"
8
9 df = spark.read.option("header", "true").csv(INPUT)
10
11 df = (df
12     .withColumn("number_of_trips_passing_wait", col("number_of_trips_passing_wait").cast("double"))
13     .withColumn("number_of_scheduled_trips", col("number_of_scheduled_trips").cast("double"))
14 )
15
16 result = (df.groupBy("borough")
17     .agg(
18         avg("number_of_trips_passing_wait").alias("avg_trips_passing_wait"),
19         avg("number_of_scheduled_trips").alias("avg_scheduled_trips"),
20         sum("number_of_trips_passing_wait").alias("sum_trips_passing_wait"),
21         sum("number_of_scheduled_trips").alias("sum_scheduled_trips")
22     )
23     .withColumn(
24         "efficiency_ratio",
25         when(col("sum_scheduled_trips") > 0, col("sum_trips_passing_wait") / col("sum_scheduled_trips"))
26         .otherwise(None)
27     )
28 )
29
30 result.write.mode("overwrite").option("header", "true").csv(OUTPUT)
31 spark.stop()
```

Figure 8. borough_aggregation.py

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, sum, when, desc
3
4 spark = SparkSession.builder.appName("route_efficiency_scoring").getOrCreate()
5
6 INPUT = "s3://pdcbucket/raw/"
7 OUTPUT = "s3://pdcbucket/processed/route_efficiency_scoring/"
8
9 df = spark.read.option("header", "true").csv(INPUT)
10
11 df = (df
12     .withColumn("number_of_trips_passing_wait", col("number_of_trips_passing_wait").cast("double"))
13     .withColumn("number_of_scheduled_trips", col("number_of_scheduled_trips").cast("double"))
14 )
15
16 route = (df.groupBy("route_id")
17     .agg(
18         sum("number_of_trips_passing_wait").alias("sum_trips_passing_wait"),
19         sum("number_of_scheduled_trips").alias("sum_scheduled_trips")
20     )
21     .withColumn(
22         "efficiency_score",
23         when(col("sum_scheduled_trips") > 0, col("sum_trips_passing_wait") / col("sum_scheduled_trips"))
24         .otherwise(None)
25     )
26     .orderBy(desc("efficiency_score"))
27 )
28
29 route.write.mode("overwrite").option("header", "true").csv(OUTPUT)
30 spark.stop()
```

Figure 9. Route_efficiency_scoring.py

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, sum, when
3
4 spark = SparkSession.builder.appName("congestion_delay_hotspots").getOrCreate()
5
6 INPUT = "s3://pdcbucket/raw/"
7 OUTPUT = "s3://pdcbucket/processed/congestion_delay_hotspots/"
8
9 # Threshold: lower ratio = worse performance (adjust if needed)
10 THRESHOLD = 0.75
11
12 df = spark.read.option("header", "true").csv(INPUT)
13
14 df = (df
15     .withColumn("number_of_trips_passing_wait", col("number_of_trips_passing_wait").cast("double"))
16     .withColumn("number_of_scheduled_trips", col("number_of_scheduled_trips").cast("double"))
17 )
18
19 agg = (df.groupBy("borough", "period")
20     .agg(
21         sum("number_of_trips_passing_wait").alias("sum_trips_passing_wait"),
22         sum("number_of_scheduled_trips").alias("sum_scheduled_trips")
23     )
24     .withColumn(
25         "efficiency_ratio",
26         when(col("sum_scheduled_trips") > 0, col("sum_trips_passing_wait") / col("sum_scheduled_trips"))
27         .otherwise(None)
28     )
29     .withColumn(
30         "is_hotspot",
31         when(col("efficiency_ratio").isNotNull() & (col("efficiency_ratio") < THRESHOLD), 1).otherwise(0)
32     )
33 )
34
35 agg.write.mode("overwrite").option("header", "true").csv(OUTPUT)
36 spark.stop()
```

Figure 10. congestion_delay_hotspots.py

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, sum, when
3 from pyspark.sql.window import Window
4 from pyspark.sql.functions import avg
5
6 spark = SparkSession.builder.appName("temporal_trend_analysis").getOrCreate()
7
8 INPUT = "s3://pdcbucket/raw/"
9 OUTPUT = "s3://pdcbucket/processed/temporal_trend_analysis/"
10
11 df = spark.read.option("header", "true").csv(INPUT)
12
13 df = (df
14     .withColumn("month", col("month").cast("int"))
15     .withColumn("number_of_trips_passing_wait", col("number_of_trips_passing_wait").cast("double"))
16     .withColumn("number_of_scheduled_trips", col("number_of_scheduled_trips").cast("double"))
17 )
18
19 monthly = (df.groupBy("borough", "month")
20     .agg(
21         sum("number_of_trips_passing_wait").alias("sum_trips_passing_wait"),
22         sum("number_of_scheduled_trips").alias("sum_scheduled_trips")
23     )
24     .withColumn(
25         "efficiency_ratio",
26         when(col("sum_scheduled_trips") > 0, col("sum_trips_passing_wait") / col("sum_scheduled_trips"))
27         .otherwise(None)
28     )
29 )
30
31 # Rolling 3-month average with standard deviation
32 w = Window.partitionBy("borough").orderBy(col("month")).rowsBetween(-2, 0)
33 monthly = monthly.withColumn("rolling_avg_efficiency", avg("efficiency_ratio").over(w))
34
35 monthly.write.mode("overwrite").option("header", "true").csv(OUTPUT)
36 spark.stop()
```

Figure 11. temporal_trend_analysis.py

Add step

Step settings

Type

- ☐ Custom JAR
- ☐ Hive program
- ☐ Shell script
- ☒ Spark application

Name

Enter a name

Deploy mode

- ☒ Cluster mode
- ☐ Client mode

Application location

Path to JAR with your application and dependencies (Client deploy mode only supports a local path).

s3://s3bucket/scripts/eta_sample_app.py

Spark-submit options - optional

Specify other options for spark-submit.

-kms, org.apache.spark.examples.SparkR

Monitoring configuration - optional

Log Amazon S3 location - optional

Log Amazon S3 location - optional

AWS KMS key - optional

Choose an AWS KMS key or enter an ARN

Step action

Action if step fails

- ☒ Continue
- ☐ Cancel and wait
- ☐ Terminate cluster

Figure 12. Configuration of Spark application step in Amazon EMR using a PySpark script stored in Amazon S3.

The YARN ResourceManager interface was used to monitor the execution of multiple Spark applications submitted to the Amazon EMR cluster. As shown in the application history view, four Spark jobs which are borough-level aggregation, route efficiency scoring, congestion delay hotspot detection, and temporal trend analysis were executed sequentially. All applications completed with a finished state and a succeeded final status, indicating successful execution without runtime failures. Individual application pages further confirm stable execution with short elapsed times (approximately 12–15 seconds per job), demonstrating efficient distributed processing and effective resource scheduling by YARN.

All Applications

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Progress	Tracking URL
application_1767097416404_0004	hadoop	temporal_trend_analysis.py	SPARK	s-06680282d2g4qbtpx	root.default	0	Tue Dec 30 20:25:10 +0800 2025	Tue Dec 30 20:26:10 +0800 2025	Tue Dec 30 20:26:23 +0800 2025	FINISHED	SUCCEEDED		History
application_1767097416404_0003	hadoop	congestion_delay_hotspots.py	SPARK	s-06905461dpgc4y14uu	root.default	0	Tue Dec 30 20:25:44 +0800 2025	Tue Dec 30 20:25:44 +0800 2025	Tue Dec 30 20:25:57 +0800 2025	FINISHED	SUCCEEDED		History
application_1767097416404_0002	hadoop	route_efficiency_scoring.py	SPARK	s-029875933c820umoxc4	root.default	0	Tue Dec 30 20:25:18 +0800 2025	Tue Dec 30 20:25:18 +0800 2025	Tue Dec 30 20:25:30 +0800 2025	FINISHED	SUCCEEDED		History
application_1767097416404_0001	hadoop	borough_aggregation.py	SPARK	s-00365921bdy9lg2cpl	root.default	0	Tue Dec 30 20:24:48 +0800 2025	Tue Dec 30 20:24:48 +0800 2025	Tue Dec 30 20:25:04 +0800 2025	FINISHED	SUCCEEDED		History

Showing 1 to 4 of 4 entries

Application application_1767097416404_0001

Application Overview

User:	hadoop
Name:	borough_aggregation.py
Application Type:	SPARK
Application Tags:	s-00365921bdy9lg2cpl
Application Priority:	0 (higher integer value indicates higher priority)
YarnApplicationState:	FINISHED
Queue:	root.default
FinalStatus Reported by AM:	SUCCEEDED
Started:	Tue Dec 30 12:24:48 +0800 2025
Launched:	Tue Dec 30 12:24:48 +0800 2025
Finished:	Tue Dec 30 12:25:04 +0800 2025
Elapsed:	15sec
Tracking URL:	History
Unmanaged Application:	false
Application Node Label expression:	-Not set-
AM container Node Label expression:	DEFAULT_PARTITION

Show 20 entries

Attempt ID	Started	Node	Logs
attempt_1767097416404_0001_000001	Tue Dec 30 20:24:48 +0800 2025	http://ip-172-31-5-60.ec2.internal:8042	Logs

Figure 13. Overview of Spark Applications and Detailed Execution of a Job in Amazon EMR

The execution performance of the Spark aggregation job was monitored using the YARN ResourceManager interface. The application completed successfully with a total execution time of approximately 12 seconds, indicating efficient parallel processing for the given dataset size. Resource utilization metrics reported by YARN showed controlled allocation of CPU and memory resources, with no container preemption observed during execution. This demonstrates effective scheduling and workload distribution across the EMR cluster nodes.

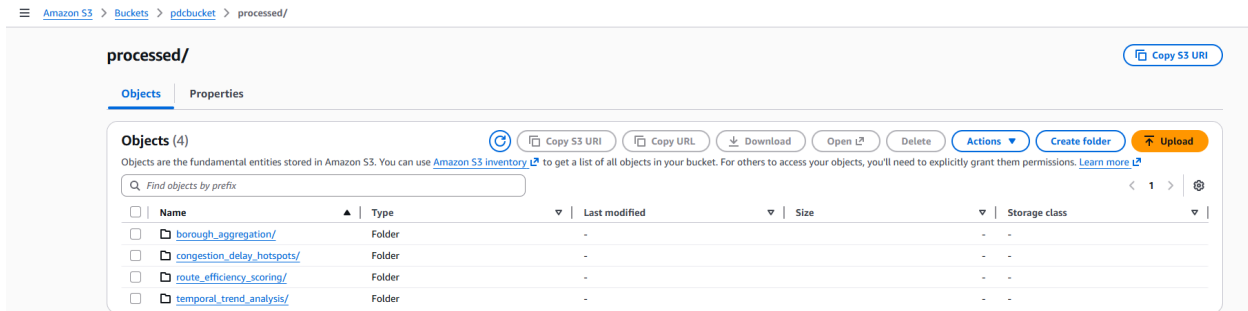


Figure 14. Processed data generated by Spark on EMR and persisted to Amazon S3 for downstream analytics.

4.3 Data Analytics and Visualisation

I. Analytical Query Layer (AWS Athena)

Due to cost constraints within the AWS Learner Lab environment, which provides a limited credit allocation of approximately USD 50, Amazon Redshift was not deployed in this implementation. Redshift requires persistent cluster provisioning, which may incur continuous costs exceeding the available learner credits. To avoid premature resource exhaustion while maintaining analytical capabilities, a serverless alternative was selected. Athena allows direct querying of processed datasets stored in Amazon S3 using standard SQL.

Before creating Athena databases and executing SQL queries, an Amazon S3 query result location was configured. Amazon Athena requires a designated S3 path to store query execution outputs, metadata, and temporary files generated during query processing.

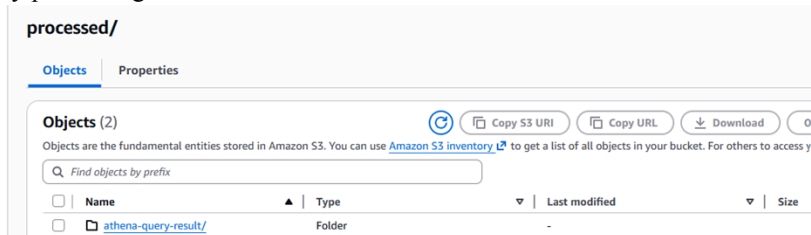


Figure 15. Creation of designated Athena S3 path

In this implementation, a dedicated folder named `athena-query-result/` was created under the processed data directory in Amazon S3. This location was specified in the Athena query settings as the default output path for all query results. Configuring this storage location is a mandatory prerequisite for running Athena queries, as Athena does not retain query outputs internally.

Following that, two logical Athena databases were implemented to separate data staging and analytical consumption concerns. The first database, `my_pdc_db`, functions as a source or staging database. This database contains external tables directly mapped to the processed datasets generated by the Amazon EMR Spark jobs and stored in Amazon S3. Its primary role is to provide structured access to raw aggregated results, such as borough-level statistics and operational performance metrics, without modifying the original processed outputs. This design preserves data lineage and allows reprocessing or re-querying of Spark outputs when required.

The second database, `pdc_datamart`, serves as an analytical data mart optimized for reporting and visualization purposes. Derived table views were created within this database. These view tables represent outputs, designed for direct consumption by external business intelligence tools, such as Power BI.

After the Athena databases were created, external tables were defined to provide structured access to multiple processed datasets generated by the Spark jobs and stored in Amazon S3. For each dataset, the table schema was explicitly specified to match the corresponding aggregated outputs, including relevant identifiers and computed performance metrics.

Each external table was mapped directly to its respective S3 directory containing Spark-generated CSV files. Header rows were excluded during table creation to ensure accurate query execution. This approach enables SQL-based analysis of distributed processing results without data duplication, allowing multiple analytical tables to be queried efficiently and consistently.

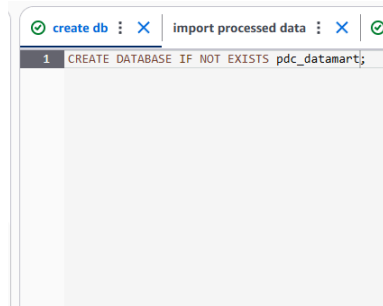


Figure 16. Creation of databases

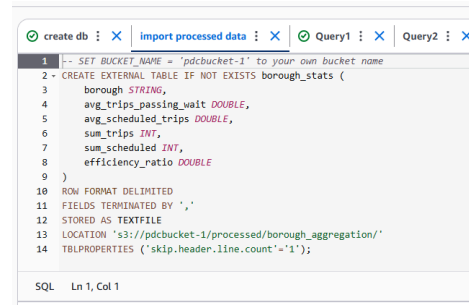


Figure 17. Athena External Table Definition

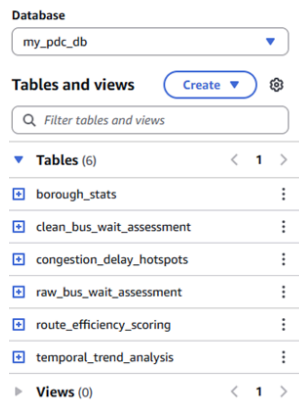


Figure 18. Athena Staging Database Table

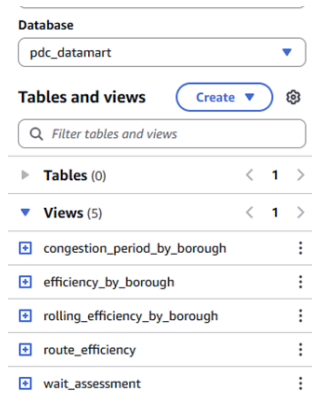


Figure 19. Athena Analytical Datamart Views

II. Visualisation Layer (Power BI)

Due to access limitations in Amazon QuickSight, we could not proceed with our initial plan. One of the main causes is that the AWS Learner Lab environment uses account-level subscriptions, and students are given temporary IAM roles, which impose access restrictions. As an alternative, Power BI was adopted for data visualization. This solution still preserves original system architecture and demonstrates flexibility in tool selection since restrictions affecting AWS-managed services do not apply to external sources like Microsoft Power BI.

The connection between Amazon Athena with Power BI was configured using an ODBC driver installed on the local machine. A system-level Data Source Name (DSN) acts as a bridge between Power BI and Athena, enabling SQL-based data retrieval. Figure below shows the DSN configuration.

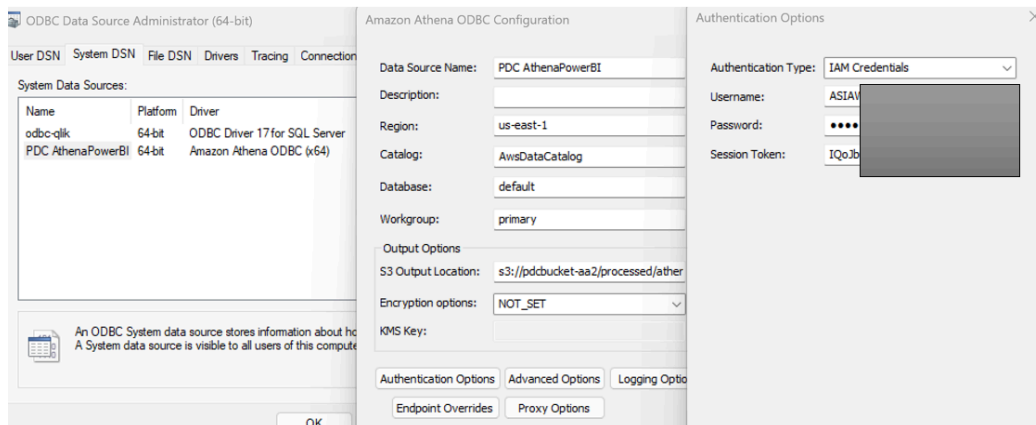


Figure 20. ODBC AWS Athena Configuration

The authentication mechanism for accessing AWS services was configured using temporary access credentials provided by AWS CLI within the AWS Learner Lab environment. The Amazon Athena ODBC driver was configured using IAM Credentials with 'Username' field represents 'aws_access_key_id', 'Password' field as 'aws_secret_access_key' and 'Session Token' as 'aws_session_token'.

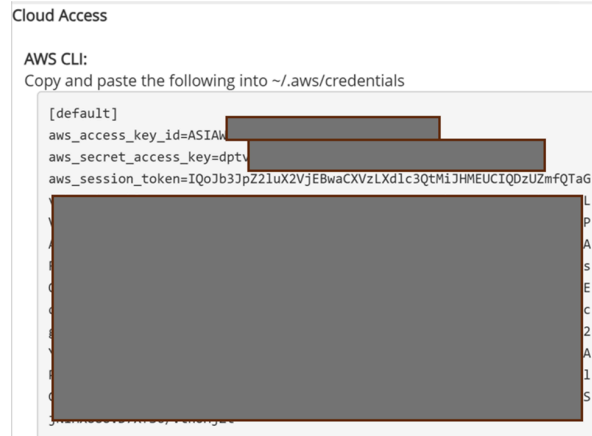


Figure 21. AWS CLI

In Power BI desktop, Get Data → ODBC option was selected to initiate the connection. Athena data mart views were selected and loaded into Power BI by using import mode to allow offline dashboard interaction. This can also avoid dependency on the AWS Learner Lab environment which is limited to a maximum of 4 hours per session used. Figure below shows the data mart views loaded into Power BI.

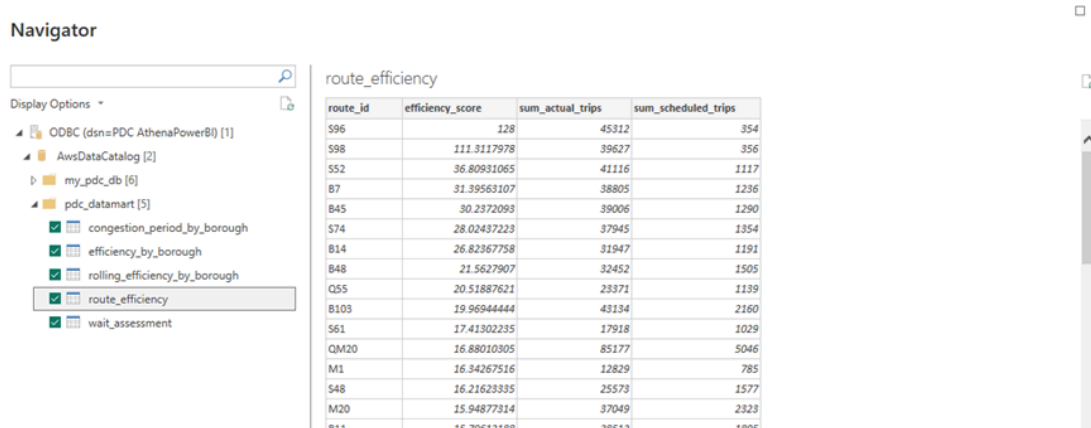


Figure 22. Data Mart Views Loaded into PowerBI

The views encapsulated pre-aggregated and analysis-ready datasets which serve each chart on the dashboard. Table 2 shows the data mart referencing each chart in the dashboard created in Power BI in Figure 23.

Table 2: Chart with Referencing Data Mart

Chart Name	Referencing Data Mart
Max Efficiency Ratio	efficiency_by_borough
Avg Waiting Time (%)	wait_assessment
Wait-Time Trend	wait_assessment
Worst Congestion Rank (Hotspot)	congestion_period_by_borough

Efficiency vs Smooth Efficiency Comparison	rolling_efficiency_by_borough
Demand vs Capacity Analysis	route_efficiency
Peak vs Off-Peak Efficiency	congestion_period_by_borough

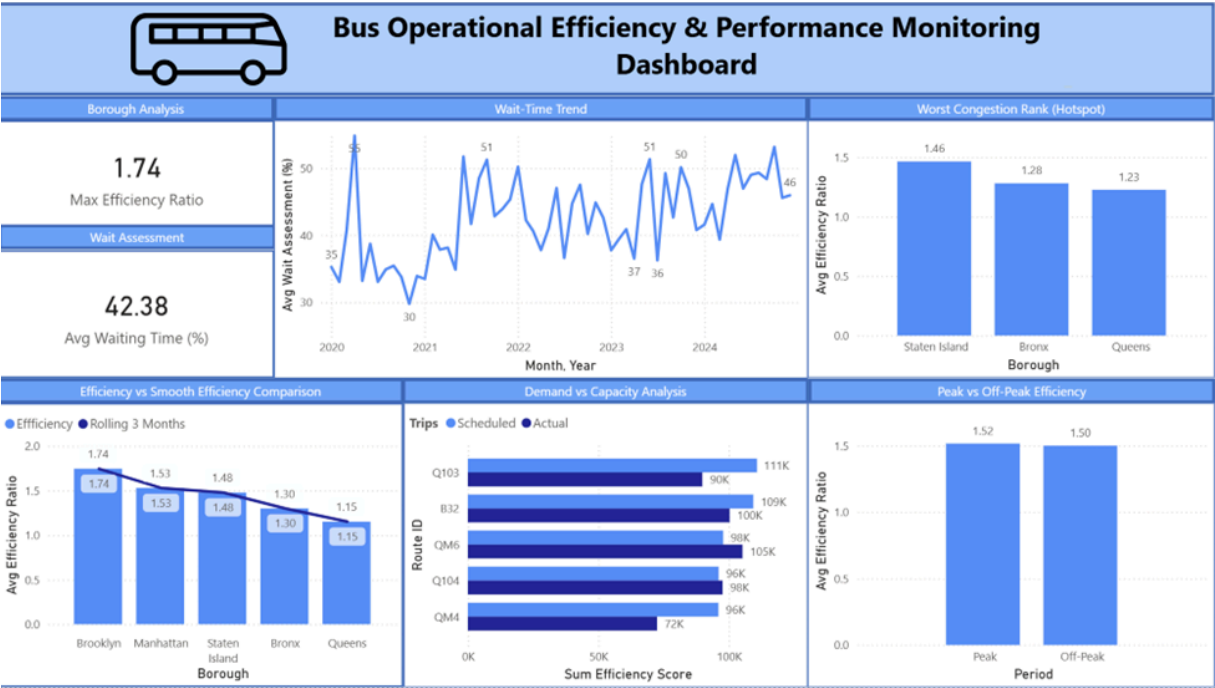


Figure 23. Overall Dashboard

5.0 Results & Discussion

The implementation of the distributed architecture successfully processed the NYC MTA dataset, with Spark jobs completing in approximately 12–15 seconds on average, demonstrating the effectiveness of parallel task execution across multiple EMR worker nodes. Despite the necessary adjustments undertaken from the proposed distributed architecture due to AWS Learner Lab constraints, the core data pipeline remained intact, proving that the implemented architecture preserved the intended distributed processing capabilities in a real-world deployment environment. The results highlight the low latency and horizontal scalability required for high-velocity real-time transit data.

From an analytical perspective, the generated dashboard revealed meaningful operational insights. Each component in the dashboard contributes to a specific analytical outcome and insights, explained in Table 3 below.

Table 3: Dashboard Outcomes and Insights

Chart	Outcomes	Insights
Max Efficiency Ratio	The maximum efficiency ratio across boroughs is 1.74, indicating that passenger demand exceeded schedule capacity by 74%.	This highlights significant demand-capacity imbalance in certain boroughs, suggesting that boroughs with high efficiency should be prioritized for capacity reallocation.
Avg Waiting Time (%)	The average wait-time assessment is 42.8%, indicating the trips meeting acceptable waiting time thresholds.	Value below 50% indicates moderate reliability, and this KPI serves as a high-level indicator for monitoring overall service reliability.
Wait-Time Trend	Noticeable fluctuations from 2020 to 2024, with some peaks and	These variations suggest that temporal factors like demand growth, operational changes, or external

	declines.	disturbances may have an impact on service reliability.
Worst Congestion Rank (Hotspot)	Staten Island, Bronx, and Queens have the worst congestion.	These boroughs represent congestion hotspots and should be prioritized in improving operational strategies.
Efficiency vs Smooth Efficiency Comparison	The comparison between raw efficiency and rolling three months efficiency shows smoother trends across boroughs.	Temporary anomalies and sustained performance issues can be distinguished, thus supporting more informed decision-making.
Demand vs Capacity Analysis	Actual demands for some routes, such as QM6 and Q104, exceed scheduled trips.	The imbalance between scheduled capacity and actual demand proves that congestion is driven by under-scheduling.
Peak vs Off-Peak Efficiency	Peak-period efficiency ratio is slightly higher than off-peak.	Congestion and inefficiency intensify during peak hours, when passenger demand is high.

Furthermore, the seamless integration between Amazon Athena's serverless query engine and Power BI validated the architecture's ability to decouple storage from compute, allowing cost-effective and interactive reporting without the need for persistent data warehousing clusters.

From a distributed computing perspective, the observed execution time reflects efficient workload distribution across Spark executors, reducing bottlenecks typically associated with single-node systems. By utilizing horizontal scaling, the architecture is capable of maintaining low processing latency as data volume increases. However, this evaluation was conducted within the constraints of the AWS Learner Lab environment, where cluster size and dataset volume were limited and real-time streaming ingestion was simulated through batch uploads. As a result, while the system demonstrates scalability in principle, further evaluation using larger datasets and live streaming services would be required to fully validate real-time performance under production-scale workloads.

6.0 Conclusion

The project successfully demonstrated and validated a scalable distributed computing framework for analysing large-scale public transportation data using the AWS Learner Lab environment. By leveraging Amazon S3 for data lake storage, EMR for parallel processing, and Athena for analytics, the architecture achieved the objectives of fault tolerance, scalability, and high-performance data handling that traditional (single-node) systems lack. The successful execution of aggregation and route efficiency scoring algorithms proved that high-frequency, complex real-time transit metrics can be processed rapidly to support timely decision-making. Conclusively, this study demonstrates that cloud-based distributed systems are essential for modernizing urban transit operations, offering a viable path for future explorations and enhancements for better urban congestion management.

References

Czarnul, P., Antal, M., Baniata, H., Griebler, D., Kertesz, A., Kessler, C. W., ... & Rakić, G. (2025). Optimization of resource-aware parallel and distributed computing: a review. *Journal of Supercomputing*, 81(7).

Hoque, R., Sharif, N. B., & Hoque, M. (2024). Distributed Parallel and Cloud Computing: A Review. *International Journal of Computer Applications*, 975, 8887.

Hassan, M., Mahin, H., Nafees, A., Paul, A., & Shrabana, S. (2025). Big data applications in intelligent transport systems: A bibliometric analysis and review. *Discover Civil Engineering*, 2, Article 00205. <https://doi.org/10.1007/s44290-025-00205-z>

Merced, A. (10 October, 2024). What is Three-Tier Data (Bronze, Silver, Gold) and How Dremio Simplifies It. Retrieved 2 December, 2025, from <https://medium.com/data-engineering-with-dremio/what-is-three-tier-data-bronze-silver-gold-and-how-dremio-simplifies-it-dd56f7fa37e4>